

Module 02a — Seeing the Topology (for GNS3 Users)

FOR
DUMMIES

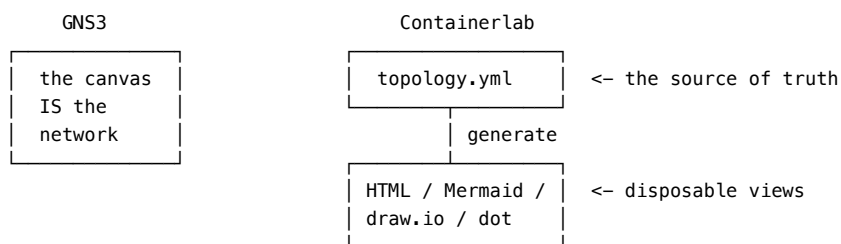
Goal: stop missing the GNS3 canvas. Learn the four ways Containerlab renders a topology, when to reach for each, and the mental shift that makes the whole thing click.

Slot this in right after [Module 02](#) — you’ll want the testlab or spine-leaf fabric handy to try things against.

1. The mental shift (this is the actual lesson)

In GNS3, the *canvas is the network*. You drag a router onto a drawing, draw a line to another router, and that act of drawing creates the topology. The picture and the reality are the same object. Which is lovely — until you want to code-review a change, or diff two versions, or hand a colleague “the network” without handing them a proprietary project file.

In Containerlab, the **YAML is the network**, and pictures are generated views of it. You never draw. You describe, then render.



The payoff is one you’ll feel within a week: **the diagram can never be out of date**. Everyone has seen a Visio in a customer’s SharePoint that stopped matching reality three changes ago. That failure mode is structurally impossible here — the diagram is regenerated from the same file that builds the lab. If they disagree, the diagram is just stale by one command.

The cost is that you can’t *design* by dragging. You design by typing. For a 4-node fabric that’s a wash; for a 40-node fabric, typing wins outright, because your topology becomes copy-paste-able and loop-able rather than 400 mouse drags.

2. The four views

View 1 — The table (inspect)

The fastest answer to “what’s running and where do I SSH?”

```
./scripts/02-deploy-lab.sh topologies/spine-leaf.clab.yml inspect
```

Node names, kinds, state, management IPv4/IPv6. This is your `show ip int brief` of the lab. Not a picture, but 80% of the time it’s what you actually wanted.

View 2 — Mermaid (diagram-as-code) ★ best fit for Aston docs

```
containerlab graph --mermaid -t topologies/spine-leaf.clab.yml
```

Writes a `.mermaid` file — a *text* description of the diagram:

```
graph TD
  spine1 --- leaf1
  spine1 --- leaf2
  ...
```

Control the layout direction with `--mermaid-direction`, which accepts TB, TD, BT, RL, and LR. For a spine-leaf fabric, TB (top-to-bottom) puts spines above leaves and looks like the design actually is:

```
containerlab graph --mermaid --mermaid-direction TB -t topologies/spine-leaf.clab.yml
```

Why this one matters most for us: Mermaid renders natively in GitHub, GitLab, Notion — and in Obsidian, which is where a lot of Aston engineering notes already live. Paste the output into a fenced ````mermaid` block in a note and you get a live diagram that lives in version control as text, diffs like text, and never needs re-exporting. A topology diagram in a markdown MOP that regenerates from the lab definition is a genuinely nice thing to hand a customer or a teammate.

View 3 — The interactive HTML graph (closest to the GNS3 canvas)

```
containerlab graph -t topologies/spine-leaf.clab.yml
```

With no format flag, `graph` starts an embedded web server on port 50080 and serves an interactive topology map — drag nodes around, see the links, get a feel for the shape.

Practical note for our setup: the lab VM is routable from Aston WiFi and the VPN, so just browse to `http://<vm-ip>:50080`. If your VM ever *isn’t* directly reachable (a locked-down segment, or a home Proxmox box behind a firewall), tunnel it over the SSH session you already have:

```
# from your laptop
ssh -L 50080:localhost:50080 you@<vm-ip>
# then browse to http://localhost:50080
```

That port-forward trick generalizes to anything else you run on the VM later, so it’s worth having in your fingers.

Note this is a *viewer*, not an editor — moving nodes changes the picture, not the lab. If you want to edit graphically, that’s View 5.

View 4 — draw.io / diagrams.net (the customer deliverable)

```
containerlab graph --drawio -t topologies/spine-leaf.clab.yml
```

This produces a `.drawio` file you can open in diagrams.net and edit like any other diagram — which makes it the right choice when the artifact needs to leave the lab: a design doc, an as-built, a slide for

a customer readout.

Under the hood it pulls and runs the `clab-io-draw` container, so it needs Docker and internet access the first time (both of which you have). You can pass options through to that tool:

```
containerlab graph --drawio \  
  --drawio-args="--theme nokia_dark --layout horizontal" \  
  -t topologies/spine-leaf.clab.yml
```

There's also `--dot` if you'd rather have Graphviz format for a scripted pipeline.

View 5 — The VS Code extension (the real GNS3 replacement)

If what you actually miss is *drawing*, install the [Containerlab VS Code extension](#). It's the closest thing to GNS3's experience, and it's built by the Containerlab community rather than bolted on:

- **TopoViewer** — a graphical topology creator/editor. YAML on one side, canvas on the other, drag nodes, and the YAML updates. This is drag-and-drop topology building with a text file as the output.
- Tree views of your local (undeployed) and running labs
- Deploy/destroy from the UI
- **Right-click a lab** → SSH to all nodes in VS Code's multi-tab terminal — a real time-saver on a fabric
- Inspect the lab in the same table format as the CLI, with a filter box
- Export the canvas to SVG for docs

If someone on the team is bouncing off the CLI-only workflow, point them here first. It's a legitimate on-ramp, not a crutch — the YAML it produces is the same YAML, so they can graduate to the CLI whenever.

Bonus: filtering big topologies

Once a fabric gets large, graphing all of it is noise. `--node-filter` takes a comma-separated list of node names and graphs only those plus their links:

```
containerlab graph --mermaid --node-filter spine1,leaf1,leaf2 \  
  -t topologies/spine-leaf.clab.yml
```

Useful for “just show me this pod” diagrams in a troubleshooting writeup.

3. Which one, when

Situation	Reach for
“What's the mgmt IP of leaf2?”	inspect

Situation	Reach for
"Is this cabled the way I think?"	<code>--mermaid</code> , eyeball it
Documenting a lab in Obsidian / a markdown MOP	<code>--mermaid</code> into a fenced block
Explaining the fabric to someone over a screenshare	HTML graph on :50080
Diagram going into a customer doc or slide	<code>--drawio</code>
"I want to <i>build</i> topologies visually"	VS Code extension + TopoViewer
Feeding a diagram into a scripted pipeline	<code>--dot</code>

4. The workflow this enables

Here's the loop that replaces GNS3's "open project, look at canvas":

1. edit `topologies/<lab>.clab.yml` (change the network)
2. `containerlab graph --mermaid -t ...` (check you drew what you meant)
3. `destroy + deploy` (make it real)
4. `link-inventory.sh` (repoint automation – module 00!)
5. paste the mermaid into your notes (documentation, for free)

Step 2 is the one GNS3 users under-use. Rendering the diagram *before* deploying is a cheap sanity check on a topology edit — you'll catch "oops, I cabled both leaf uplinks to spine1" in two seconds instead of after a four-node boot and a confused OSPF troubleshooting.

Step 5 is the one that makes people converts. Documentation that generates itself from the source of truth is a genuinely better situation than any GUI screenshot workflow.

5. Try it

Against the spine-leaf fabric from [Module 03](#) (deploy it if it isn't up):

1. Run `inspect` and note the four management IPs.
2. Generate a Mermaid graph with `--mermaid-direction TB`. Read the text output *before* rendering it — you should be able to see the full mesh in the link list.
3. Paste it into a markdown file in Obsidian (or a GitHub gist) inside a ````mermaid` block. Confirm it renders.
4. Start the HTML graph and open it from your laptop. If it doesn't load directly, do it again through the SSH port-forward — get that working at least once so you know the trick.
5. Generate a `.drawio` and open it in [diagrams.net](#).

6. **The proof:** add a `leaf3` node and its two spine links to a *copy* of the topology file, regenerate the Mermaid without deploying anything, and watch the diagram show a change that doesn't exist yet in the running lab. Sit with that for a second — the diagram tracks the *intent* file, and reality is what you sync to it.

Checkpoint

You can produce all four view formats from one topology file

A Mermaid diagram of your fabric is rendering in your notes

You've reached the HTML graph from your laptop, at least once via the SSH port-forward

You can explain why a generated diagram can't go stale the way a Visio does

What you learned

That “I can't see my topology” was really “I'm used to the picture being the source of truth.” Containerlab inverts that: one text file, many disposable views, none of which can drift from it. Plus the practical kit — *inspect* for facts, Mermaid for docs, the HTML graph for talking to humans, draw.io for deliverables, and the VS Code extension when you genuinely want to drag things around.

Back to: [Module 03 — The spine-leaf fabric](#)